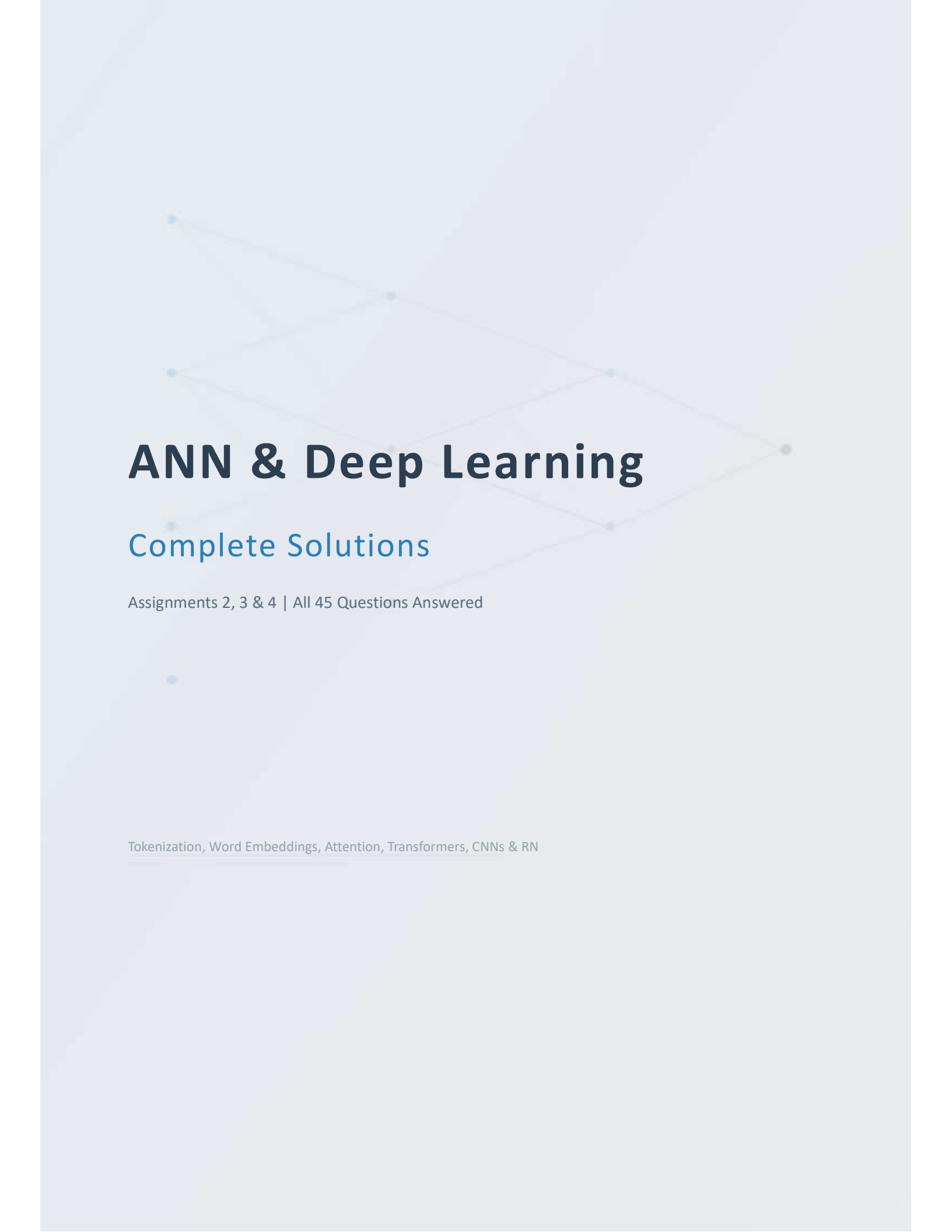




ANN & Deep Learning

Complete Solutions

Assignments 2, 3 & 4 | All 45 Questions Answered

Tokenization, Word Embeddings, Attention, Transformers, CNNs & RN

Assignment 4: Tokenization, Word Embeddings, Attention & Transformers

Q1. Why can neural networks not directly process raw text? Explain the purpose of Tokenization, Vocabulary, Token IDs, and the difference between character-level and word-level tokenization.

Neural networks cannot directly process raw text because they operate on numerical computations. Text is categorical and discrete, whereas neural networks require continuous numerical inputs (tensors) to perform matrix multiplications and gradient-based optimization.

Tokenization: It is the process of splitting raw text into smaller units called tokens. These tokens can be characters, words, or subwords. Tokenization converts unstructured text into a structured format that can be numerically encoded.

Vocabulary: The vocabulary is the set of all unique tokens extracted from the training corpus. Each token in the vocabulary is assigned a unique index. The vocabulary size determines the dimensionality of the input representation.

Token IDs: Token IDs are integer indices that map each token to a specific position in the vocabulary. For example, if 'learning' is the 342nd word in the vocabulary, its token ID is 342. These IDs serve as lookup indices for embedding layers.

Character-level vs Word-level Tokenization:

- Character-level tokenization splits text into individual characters. It has a small vocabulary size and handles unknown words well, but produces very long sequences and makes it harder to capture semantic meaning.
- Word-level tokenization splits text into words. It preserves semantic meaning better and produces shorter sequences, but requires a large vocabulary and cannot handle out-of-vocabulary (OOV) words.

Q2. Given the sentence "Deep learning models learn patterns" with vocabulary mapping, (a) Convert to token IDs, (b) Embedding matrix shape for embedding dim=4, (c) Why embeddings over one-hot?

Given Vocabulary: Deep=1, learning=2, models=3, learn=4, patterns=5

(a) Token IDs:

Sentence: "Deep learning models learn patterns"

Token IDs: [1, 2, 3, 4, 5]

(b) Embedding Matrix Shape:

Sentence length = 5 words

Embedding dimension = 4

Embedding matrix shape = (5, 4) or [5 x 4]

Each of the 5 words is represented by a 4-dimensional dense vector. The embedding layer looks up each token ID and retrieves its corresponding 4-dimensional vector from the embedding matrix.

(c) Why embeddings are preferred over one-hot vectors:

- Dimensionality: One-hot vectors have dimension equal to vocabulary size (e.g., 50,000), while embeddings use a fixed small dimension (e.g., 128-1024).
- Semantic capture: Embeddings place similar words close together in vector space; one-hot vectors have no semantic relationship between words.
- Memory efficiency: One-hot matrices are extremely sparse, wasting memory. Embeddings are dense and compact.
- Generalization: Embeddings allow the model to generalize across similar words, whereas one-hot encoding treats every word independently.

Q3. What are word embeddings? Explain how embeddings capture semantic similarity and contextual meaning. Why are embeddings better than one-hot encoding?

Word embeddings are dense, low-dimensional, continuous vector representations of words. Each word is mapped to a real-valued vector in a high-dimensional space (typically 50 to 1024 dimensions). These vectors are learned during training and encode semantic and syntactic information about words.

How embeddings capture semantic similarity: Words with similar meanings are mapped to nearby points in the vector space. For example, the vectors for 'king' and 'queen' will be closer to each other than to 'apple'. This is measured using cosine similarity or Euclidean distance. Mathematically, if v_{king} and v_{queen} are embedding vectors, their cosine similarity is: $\cos(\theta) = (v_{\text{king}} \cdot v_{\text{queen}}) / (||v_{\text{king}}|| * ||v_{\text{queen}}||)$.

How embeddings capture contextual meaning (in contextual embeddings like BERT): Traditional embeddings assign one fixed vector per word. Contextual embeddings generate different vectors depending on the surrounding words. For example, 'bank' in 'river bank' gets a different vector than 'bank' in 'financial bank' because the model attends to different context words.

Why embeddings are better than one-hot encoding:

- One-hot vectors are orthogonal (all have 90-degree angle between them), meaning no similarity information is captured.
- One-hot encoding requires $O(V)$ memory per word where V is vocabulary size; embeddings use $O(d)$ where d is embedding dimension ($d \ll V$).
- Embeddings enable analogy solving: $v(\text{'king'}) - v(\text{'man'}) + v(\text{'woman'})$ is approximately equal to $v(\text{'queen'})$.
- Embeddings act as a lookup table that projects discrete tokens into a continuous space suitable for neural network processing.

Q4. Why is assigning one vector per word problematic for words with multiple meanings? Use "bank" as an example. How do contextual embeddings improve this?

Traditional word embeddings (like Word2Vec or GloVe) assign exactly one vector to each word in the vocabulary, regardless of context. This is problematic for polysemous words - words with multiple meanings - because the single vector must somehow encode all possible senses of the word simultaneously.

Example - "bank":

- River bank: The sloping edge of a river. Context words: water, flow, edge, land.
- Financial bank: An institution for depositing and lending money. Context words: money, account, loan, interest.

In traditional embeddings, both meanings share the same vector representation. This causes the vector to be an average of both contexts, making it ambiguous and suboptimal for downstream tasks.

How contextual embeddings improve this: Contextual embeddings (from models like ELMo, BERT, GPT) generate dynamic representations that change based on the surrounding text. The model processes the entire sentence and produces context-aware vectors. For 'bank' in 'I sat by the river bank', the model attends to 'river' and produces a nature-related vector. For 'I went to the bank to deposit money', it attends to 'deposit' and 'money' and produces a finance-related vector.

Q5. Explain the intuition behind the attention mechanism. Why was attention introduced? How does attention help models focus on important words?

The attention mechanism is inspired by human visual attention - when we look at a scene, we focus on specific parts while ignoring others. Similarly, in neural networks, attention allows the model to selectively focus on the most relevant parts of the input when producing each part of the output.

Why attention was introduced: Traditional sequence-to-sequence models (encoder-decoder with RNNs) compress all input information into a single fixed-size context vector. For long sequences, this becomes an information bottleneck - the model cannot retain all relevant details. Attention was introduced to allow the decoder to 'look back' at the entire input sequence at every step and focus on the most relevant parts.

How attention helps focus on important words:

- Attention computes a score (or alignment score) between each input position and the current output position being generated.
- Higher scores indicate greater relevance. These scores are passed through a softmax function to create a probability distribution (attention weights).
- The context vector is computed as a weighted sum of all input representations, where the weights are the attention scores.
- Formula: $\text{context} = \sum(\alpha_i * h_i)$ where $\alpha_i = \text{softmax}(\text{score}(\text{query}, \text{key}_i))$ and h_i are the input hidden states.

This means when generating a translation for a word, the model can directly attend to the most relevant source words, even if they are far apart in the input sequence.

Q6. Suppose $Q = [1, 2]$ and $K = [2, 1]$. Compute the attention score using dot product $Q \cdot K$. What does a larger score represent? Why are query and key vectors used?

Given: $Q = [1, 2]$, $K = [2, 1]$

Attention Score Formula: $\text{Score} = Q \cdot K$ (dot product)

(a) Computation:

$$\text{Score} = Q \cdot K = (1 \times 2) + (2 \times 1) = 2 + 2 = 4$$

(b) What a larger score represents: A larger attention score indicates higher similarity or relevance between the query and the key. In the context of sequence models, it means the token represented by the query should pay more attention to the token represented by the key. The dot product measures the alignment between two vectors - when vectors point in similar directions, the dot product is large and positive.

(c) Why query and key vectors are used:

- Query (Q): Represents the current token asking 'which other tokens should I attend to?'
- Key (K): Represents each token's identity, answering 'what do I contain?'
- Value (V): Contains the actual information to be aggregated.

- The Q-K dot product determines how much each value should contribute to the output. This separation allows the model to learn different roles for matching (Q, K) versus information delivery (V).

Q7. Explain self-attention. How does a word attend to other words in the same sentence? Why is self-attention more powerful than RNN for long sentences?

Self-attention is a mechanism where each token in a sequence computes attention scores with all other tokens in the same sequence. Unlike traditional attention (between encoder and decoder), self-attention allows tokens to interact with each other within the same input sequence.

How a word attends to other words:

- For each word in the sentence, we create a Query vector (Q), Key vector (K), and Value vector (V) by multiplying the word's embedding with learned weight matrices W_Q , W_K , W_V .
- Attention scores are computed as: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$
- The softmax ensures the weights sum to 1, creating a weighted average of all value vectors.
- Example: In 'The cat sat on the mat', when processing 'sat', self-attention might assign high weights to 'cat' (subject) and 'mat' (location).

Why self-attention is more powerful than RNN for long sentences:

- Long-range dependencies: In RNNs, information from distant tokens must travel through many time steps, causing vanishing gradients. Self-attention connects any two positions directly with a single attention computation.
- Parallelization: RNNs process sequences sequentially (one token at a time). Self-attention processes all tokens simultaneously, making it much faster on modern hardware (GPUs/TPUs).
- Path length: In RNNs, the path length between distant positions is $O(n)$. In self-attention, it is $O(1)$ - every token can directly attend to every other token.
- Interpretability: Attention weights explicitly show which words influence each other, providing transparency into model decisions.

Q8. Draw and explain the basic Transformer architecture including Encoder, Decoder, Multi-head Attention, Feedforward layers, and Positional Encoding. Explain each component.

The Transformer architecture consists of two main components: the Encoder (left side) and the Decoder (right side), stacked in layers.

Encoder: The encoder processes the input sequence and converts it into a continuous representation. It consists of a stack of N identical layers (typically $N=6$ or 12). Each layer has two sub-layers: (1) Multi-head self-attention mechanism, and (2) Position-wise fully connected feed-forward network. Residual connections and layer normalization are applied around each sub-layer. The encoder's output is a set of context-rich vectors representing the input sequence.

Decoder: The decoder generates the output sequence autoregressively (one token at a time). It also has a stack of N identical layers, each with three sub-layers: (1) Masked multi-head self-attention (prevents attending to future positions), (2) Multi-head attention over the encoder's output (cross-attention), and (3) Position-wise feed-forward network. The masked attention ensures that predictions for position i can depend only on known outputs at positions less than i .

Multi-head Attention: Instead of performing a single attention function, the Transformer runs multiple attention operations (heads) in parallel with different learned linear projections. Each head can focus on different aspects of the token relationships. The outputs of all heads are concatenated and linearly projected to produce the final output. Multi-head attention allows the model to jointly attend to information from different representation subspaces.

Feedforward Layers: Each encoder and decoder layer contains a fully connected feed-forward network applied identically to each position. It consists of two linear transformations with a ReLU activation in between: $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$. This adds non-linear transformation capacity to the model and is applied position-wise (the same network is used for all positions).

Positional Encoding: Since the Transformer has no recurrent or convolutional structure, it has no inherent notion of sequence order. Positional encoding is added to the input embeddings to inject information about the position of each token in the sequence. The original Transformer uses sinusoidal functions: $\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$ and $\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$. These encodings allow the model to learn to attend by relative positions.

Q9. Transformers process all words in parallel. Why do Transformers need positional encoding? What problem occurs without positional information?

Transformers process all tokens simultaneously through self-attention, unlike RNNs which process tokens one by one in order. While this parallelization provides computational efficiency, it means the Transformer has no built-in mechanism to know the order of tokens in a sequence.

Why positional encoding is needed: Without positional information, the Transformer would be permutation-invariant - it would produce the same output regardless of word order. For example, 'Dog bites man' and 'Man bites dog' would be treated identically, which is clearly unacceptable for language understanding. Positional encoding injects position-dependent signals into the input representations so the model can distinguish between different orderings.

Problems without positional information:

- Word order ambiguity: The model cannot distinguish 'I love you' from 'You love I'.
- Loss of syntax: Grammatical structure depends on word order. Without position info, subject-verb-object relationships cannot be learned.
- Context misunderstanding: In 'The cat chased the mouse because it was hungry', knowing the position of 'it' is crucial for determining whether 'it' refers to the cat or the mouse.

Positional encodings solve this by adding a unique signal to each position, allowing the model to learn that tokens at different positions play different structural roles in the sentence.

Q10. Large language models like ChatGPT are based on Transformers. Explain why Transformers became more successful than RNNs in NLP. Discuss parallel processing, long-range dependencies, scalability, and training efficiency.

Transformers have become the dominant architecture in NLP, powering models like GPT, BERT, and T5, replacing RNNs as the go-to architecture. This shift occurred due to several fundamental advantages:

Parallel Processing: RNNs process sequences sequentially - each hidden state depends on the previous one. This means token at position t cannot be computed until all tokens before it are processed. Transformers, through self-attention, compute all token interactions simultaneously. This allows massive parallelization on GPUs/TPUs, leading to dramatically faster training times. For a sequence of length n , RNNs require $O(n)$ sequential steps, while Transformers can process the entire sequence in $O(1)$ parallel steps (though attention computation itself is $O(n^2)$).

Long-range Dependencies: In RNNs, information from distant tokens must propagate through many intermediate states, leading to vanishing gradients and information loss. The effective context window is limited. In Transformers, every token can directly attend to every other token regardless of distance. The attention mechanism creates a direct path between any two positions, making it much easier to capture long-range dependencies (e.g., coreference across paragraphs).

Scalability: Transformer performance continues to improve as model size and data increase - a trend known as scaling laws. Models with billions of parameters (GPT-3, GPT-4, PaLM) show emergent capabilities. RNNs do not scale as effectively because their sequential nature becomes a bottleneck. The Transformer architecture can be scaled up by increasing layers, heads, and dimensions while maintaining stable training dynamics.

Training Efficiency: Transformer training can be highly optimized with modern hardware. The matrix multiplication operations in attention and feedforward layers map efficiently to GPU/TPU architectures. Techniques like mixed-precision training, gradient checkpointing, and distributed training are easier to apply to Transformers than to RNNs. Additionally, the Transformer architecture enables pre-training on massive unlabeled corpora followed by fine-tuning on specific tasks, which has proven extremely effective.

Q11. Compare one-hot encoding and word embeddings in terms of vector size, semantic meaning, sparsity, and memory efficiency. Why do modern NLP systems prefer embeddings?

Property	One-Hot Encoding	Word Embeddings
Vector Size	Vocabulary size (e.g., 50,000)	Fixed small (e.g., 128-1024)
Semantic Meaning	No similarity between vectors	Captures semantic relationships
Sparsity	Extremely sparse (one '1', rest '0')	Dense (all values non-zero)
Memory Efficiency	Very inefficient for large vocab	Highly efficient

Why modern NLP systems prefer embeddings:

- Dense representations pack more information per dimension and enable efficient computation.
- Semantic relationships are preserved - similar words cluster together, enabling better generalization.
- Embeddings support transfer learning - pre-trained embeddings (Word2Vec, GloVe, FastText) can be reused across tasks.
- Lower memory footprint despite storing all vocabulary vectors in a lookup table.
- Embeddings enable arithmetic operations that capture linguistic relationships (king - man + woman = queen).

Q12. Suppose vocabulary size = 20,000 and embedding dimension = 300. Find (a) embedding matrix size, (b) trainable parameters, (c) output shape for sentence length 10.

Given: Vocabulary size $|V| = 20,000$; Embedding dimension $d = 300$

(a) Embedding Matrix Size:

Embedding Matrix E has shape: $|V| \times d = 20,000 \times 300$

(b) Number of Trainable Parameters:

Total parameters = $|V| \times d = 20,000 \times 300 = 6,000,000$

With bias (if applicable): parameters = $(|V| \times d) + (|V| \times d) = 12,000,000$

Typically, embedding layers do not include bias terms. So the answer is 6 million trainable parameters.

(c) Output Shape for Sentence Length 10:

Input: sequence of 10 token IDs -> shape (10,)

Each token ID maps to a 300-dim vector
 Output shape = (10, 300) or [10 x 300]

Q13. What is multi-head attention? Why does the Transformer use multiple attention heads instead of one? How can different heads learn different relationships?

Multi-head attention is an extension of the self-attention mechanism where the attention function is performed multiple times in parallel with different learned linear projections. Instead of computing attention once on the full-dimensional queries, keys, and values, the model projects these into h different lower-dimensional subspaces and applies attention independently in each.

Why multiple heads instead of one:

- Single attention is limited to one type of relationship. Multiple heads allow the model to capture diverse patterns simultaneously.
- Each head can specialize in different linguistic phenomena - syntax, semantics, coreference, etc.
- The concatenation of multiple heads provides a richer representation than a single attention output.

How different heads learn different relationships:

- Syntactic head: One head might learn to connect verbs to their subjects (e.g., linking 'sat' to 'cat').
- Coreference head: Another head might link pronouns to their antecedents (e.g., 'it' to 'cat').
- Semantic head: A third head might connect words with similar meanings or related concepts.
- Positional head: Some heads learn to attend to nearby tokens regardless of content.

This specialization emerges naturally during training because each head has its own set of learned projection matrices, allowing them to focus on different aspects of the input representation.

Q14. Differentiate between Transformer Encoder and Decoder. Discuss their roles in language understanding and text generation. Give one application for each.

Encoder:

- Uses bidirectional self-attention - each token can attend to all tokens in the input, both before and after it.
- Purpose: Creates rich contextualized representations of the input sequence.

- Role in language understanding: Encoders excel at understanding tasks where the full context is available - classification, named entity recognition, sentiment analysis.
- Example application: BERT (Bidirectional Encoder Representations from Transformers) - used for text classification, question answering, named entity recognition.

Decoder:

- Uses masked (causal) self-attention - each token can only attend to previous tokens and itself, not future tokens.
- Purpose: Generates output sequences autoregressively, one token at a time.
- Role in text generation: Decoders excel at generating coherent text because they learn to predict the next token given all previous tokens.
- Example application: GPT (Generative Pre-trained Transformer) - used for text generation, completion, code generation, conversational AI.

Key difference: The encoder can look at the entire input at once (bidirectional), while the decoder must generate left-to-right and can only attend to already-generated tokens (autoregressive).

Q15. Explain how Transformers are used in any three applications. Why are Transformers suitable for these tasks?

1. Machine Translation: Transformers power state-of-the-art translation systems (Google Translate, DeepL). The encoder processes the source language sentence, and the decoder generates the target language sentence. Cross-attention in the decoder allows each output token to attend to all input tokens, enabling fluent and accurate translations. Transformers capture long-range dependencies better than previous RNN-based systems, resulting in more coherent translations of long sentences.

2. Chatbots and Conversational AI: Models like ChatGPT and Claude use the Transformer decoder architecture to generate human-like responses. The self-attention mechanism allows the model to maintain context across long conversations, remembering user preferences and earlier parts of the dialogue. The parallel processing capability enables fast response generation.

3. Text Summarization: Transformers can read long documents and generate concise summaries. Models like BART and T5 use encoder-decoder architectures where the encoder comprehends the full document and the decoder produces a condensed version. Attention mechanisms help identify the most important sentences and concepts to include in the summary.

Why Transformers are suitable:

- Self-attention captures relationships between any two tokens regardless of distance.
- Pre-training on massive corpora followed by fine-tuning enables transfer learning.
- The architecture is highly parallelizable, enabling efficient training on large datasets.

- Multi-head attention captures different types of linguistic relationships simultaneously.

Assignment 3: Dense Prediction with CNNs & Recurrent Neural Networks

Part 1: Dense Prediction with CNNs

Q1. Define dense prediction. How is it fundamentally different from image classification and object detection? Give one real-world example.

Dense prediction refers to tasks where the model makes a prediction for every single pixel in the input image, producing a dense output map with the same spatial resolution as the input (or close to it). The goal is to assign a label or value to each pixel.

Difference from Image Classification: Image classification assigns a single label to an entire image (e.g., 'cat' or 'dog'). The output is one category for the whole image. In dense prediction, the output is a full map where each pixel has its own prediction.

Difference from Object Detection: Object detection identifies objects with bounding boxes and class labels. It outputs a variable number of boxes per image but does not provide pixel-level segmentation. Dense prediction provides precise pixel-level masks and boundaries.

Real-world example: Autonomous driving requires semantic segmentation (a dense prediction task) to identify which pixels belong to roads, pedestrians, vehicles, traffic signs, etc. Every pixel must be classified for safe navigation.

Q2. Standard CNNs reduce spatial resolution due to pooling. (a) Why does this happen? (b) Why is this a problem for segmentation? (c) What information is lost?

(a) Why spatial resolution is reduced: Pooling layers (max pooling, average pooling) downsample feature maps by aggregating information in local neighborhoods. A 2x2 pooling with stride 2 reduces both height and width by a factor of 2. Multiple pooling layers compound this reduction. For example, 3 pooling layers reduce resolution by $2^3 = 8x$. Convolution with stride > 1 also reduces resolution.

(b) Why this is a problem for segmentation: Segmentation requires a class label for every pixel in the original input image. If the feature map is too small (e.g., 32x32 for a 256x256 input), it becomes

extremely difficult to accurately map predictions back to individual pixels. Fine details like object boundaries and thin structures would be lost.

(c) Information lost:

- Spatial precision: Exact pixel locations of object boundaries are blurred.
- Fine details: Thin structures like hair, wires, or distant pedestrians disappear.
- Small objects: Objects smaller than the receptive field of pooling layers may be completely lost.
- Localization accuracy: The model knows what is present but not exactly where each pixel belongs.

Q3. An image of size 256 x 256 passes through 3 pooling layers (2x2, stride 2). Compute (a) final feature map size, (b) percentage of spatial resolution retained, (c) impact on pixel-level prediction.

Given: Input size = 256 x 256; Pooling: 2x2 kernel, stride 2; Number of pooling layers = 3

(a) Final Feature Map Size:

After each pooling layer, spatial dimensions are halved:

After Pool 1: $256/2 \times 256/2 = 128 \times 128$

After Pool 2: $128/2 \times 128/2 = 64 \times 64$

After Pool 3: $64/2 \times 64/2 = 32 \times 32$

Final feature map size = 32×32

(b) Percentage of Spatial Resolution Retained:

$$\begin{aligned} \text{Retained percentage} &= (32 \times 32) / (256 \times 256) \times 100 \\ &= 1024 / 65536 \times 100 = 1.5625\% \end{aligned}$$

(c) Impact on Pixel-level Prediction: With only 1.56% of the original resolution retained, pixel-level prediction becomes extremely challenging. Each feature map pixel corresponds to an 8x8 region in the original image ($256/32 = 8$). Object boundaries become blurry, small objects may disappear entirely, and precise segmentation masks cannot be produced without additional upsampling mechanisms.

Q4. Explain how a classification CNN is converted into an FCN. What happens to fully connected layers? Why does this allow variable input size?

A Fully Convolutional Network (FCN) converts a classification CNN by replacing fully connected (FC) layers with convolutional layers. This transformation was introduced in the landmark FCN paper by Long et al. (2015).

What happens to fully connected layers: An FC layer that takes a flattened input of size $H \times W \times C$ and produces N outputs can be equivalently replaced by a convolutional layer with N filters of size $H \times W$. For example, an FC layer with input $7 \times 7 \times 512$ and output 4096 is replaced by a conv layer with 4096 filters of size 7×7 . The weight values remain the same - they are just reshaped from a 2D matrix into a 4D convolutional kernel.

Why this allows variable input size: Fully connected layers require a fixed input size because they have a fixed number of weights. If the input dimensions change, the flattened vector size changes, breaking the matrix multiplication. Convolutional layers, however, can accept inputs of any spatial size. A conv filter slides over the input regardless of its dimensions. With FCNs, a larger input simply produces a larger output feature map while maintaining the same network architecture and weights.

Q5. A fully connected layer takes input of size $7 \times 7 \times 512$ and outputs 4096 neurons. (a) Convert to equivalent convolutional layer, (b) Filter size and number of filters, (c) Output dimension.

Given: FC input = $7 \times 7 \times 512$; FC output = 4096 neurons

(a) Equivalent Convolutional Layer:

```
Replace the FC layer with a convolutional layer having:
- Number of filters = 4096 (same as FC output neurons)
- Filter size =  $7 \times 7$  (same as input spatial dimensions)
- Input channels = 512
```

(b) Filter Size and Number of Filters:

```
Filter (kernel) size:  $7 \times 7 \times 512$ 
Number of filters: 4096
Total parameters:  $(7 \times 7 \times 512) \times 4096 = 102,760,448$ 
```

(c) Output Dimension:

```
Using the convolution output formula:  $\text{Output} = (W - F + 2P) / S + 1$ 
Where  $W=7$ ,  $F=7$ ,  $P=0$ ,  $S=1$ 
Output spatial size =  $(7 - 7) / 1 + 1 = 1$ 
Output dimension =  $1 \times 1 \times 4096$ 
```


Q6. Explain different upsampling methods: nearest neighbor, bilinear interpolation, and transposed convolution. Compare them on learnability, accuracy, and complexity.

Nearest Neighbor Upsampling: Each pixel is replicated to fill a larger neighborhood. For 2x upsampling, each pixel becomes a 2x2 block of identical values. It is the simplest method with no learnable parameters.

Bilinear Interpolation: Computes output pixel values as a weighted average of the four nearest input pixels based on spatial distance. Produces smoother results than nearest neighbor but still has no learnable parameters.

Transposed Convolution (Deconvolution): Uses learnable filters to upsample. It performs the reverse operation of convolution - spreading each input value across a neighborhood using a learned kernel. Unlike interpolation methods, transposed convolution has trainable parameters that can be optimized for the specific task.

Method	Learnable	Accuracy	Complexity
Nearest Neighbor	No	Low	Very Low
Bilinear Interpolation	No	Medium	Low
Transposed Conv	Yes	High	Medium

Q7. A feature map of size 16 x 16 passes through transposed convolution with stride=2, kernel=4. (a) Estimate output size, (b) Why does this increase resolution?

Given: Input = 16 x 16; Stride = 2; Kernel size = 4

(a) Output Size:

Formula for transposed convolution output:

Output size = (Input - 1) x stride + kernel_size

Output size = (16 - 1) x 2 + 4 = 15 x 2 + 4 = 30 + 4 = 34

Output dimension = 34 x 34

(b) Why this operation increases resolution: Transposed convolution works by inserting zeros between input values (determined by stride) and then applying a convolution. With stride=2, a 16x16 input effectively becomes a 31x31 grid with zeros between original values, then the 4x4 kernel convolves over this expanded grid to produce a 34x34 output. The learnable kernel fills in the interpolated values, learning the best way to upsample features for the task at hand.

Q8. Explain why skip connections are used in dense prediction models. What is combined? Why are early layer features important? Relate to localization vs semantic understanding.

Skip connections are pathways that bypass one or more layers and connect earlier layers directly to later layers. In dense prediction (e.g., U-Net), skip connections concatenate features from the encoder (downsampling path) with features from the decoder (upsampling path) at corresponding spatial resolutions.

What is combined: Features from early encoder layers (shallow, high-resolution features with precise spatial information) are concatenated with features from decoder layers (deep, semantically rich features with coarse spatial information).

Why early layer features are important: Early layers in a CNN detect low-level features like edges, corners, textures, and colors. These features contain precise spatial localization information because they have not been heavily downsampled. Deep layers detect high-level semantic features like 'car', 'person', or 'road' but have lost precise spatial coordinates due to pooling.

Localization vs Semantic Understanding trade-off: Deep networks face a trade-off: deeper layers gain semantic understanding (knowing WHAT is in the image) but lose localization ability (knowing WHERE exactly it is). Skip connections solve this by combining the best of both worlds: the decoder receives both high-resolution spatial cues from shallow layers and semantic context from deep layers. This enables precise boundary prediction while maintaining correct class labels.

Q9. For semantic segmentation with image size 128 x 128 and 4 classes: (a) Output tensor size, (b) What each pixel output represents, (c) Activation function and why.

Given: Image size = 128 x 128; Number of classes = 4

(a) Output Tensor Size:

```
Output shape = [Batch x Classes x Height x Width]
              = [Batch x 4 x 128 x 128]
```

(b) What each pixel output represents: Each pixel has 4 values (one per class). After applying softmax, these values represent the probability that the pixel belongs to each of the 4 classes. For example, $\text{output}[0, :, 50, 60] = [0.1, 0.7, 0.15, 0.05]$ means pixel at (50, 60) has 70% probability of belonging to class 1.

(c) Activation function: Softmax is applied across the class dimension for each pixel independently. It converts raw logits into probability distributions that sum to 1. Softmax is used because: (1) It

produces well-calibrated probabilities for each class, (2) It enables cross-entropy loss computation during training, (3) The argmax of the output gives the predicted class label per pixel.

Q10. Design a CNN architecture for semantic segmentation of road scenes. Include input layer, encoder, bottleneck, decoder, skip connections, and final output layer.

A U-Net style architecture for road scene segmentation:

Input Layer: Input: 512 x 512 x 3 (RGB road scene image)

Encoder (Downsampling Path):

- Block 1: Conv(3->64, 3x3) + ReLU + Conv(64->64, 3x3) + ReLU + MaxPool(2x2) -> 256x256x64
- Block 2: Conv(64->128, 3x3) + ReLU + Conv(128->128, 3x3) + ReLU + MaxPool(2x2) -> 128x128x128
- Block 3: Conv(128->256, 3x3) + ReLU + Conv(256->256, 3x3) + ReLU + MaxPool(2x2) -> 64x64x256
- Block 4: Conv(256->512, 3x3) + ReLU + Conv(512->512, 3x3) + ReLU + MaxPool(2x2) -> 32x32x512

Bottleneck: Conv(512->1024, 3x3) + ReLU + Conv(1024->1024, 3x3) + ReLU -> 32x32x1024

Decoder (Upsampling Path):

- UpBlock 4: TransposedConv(1024->512, 2x2, stride 2) + Concat with Encoder Block 4 + Conv(512->512, 3x3) + ReLU -> 64x64x512
- UpBlock 3: TransposedConv(512->256, 2x2, stride 2) + Concat with Encoder Block 3 + Conv(256->256, 3x3) + ReLU -> 128x128x256
- UpBlock 2: TransposedConv(256->128, 2x2, stride 2) + Concat with Encoder Block 2 + Conv(128->128, 3x3) + ReLU -> 256x256x128
- UpBlock 1: TransposedConv(128->64, 2x2, stride 2) + Concat with Encoder Block 1 + Conv(64->64, 3x3) + ReLU -> 512x512x64

Final Output Layer: Conv(64 -> num_classes, 1x1) + Softmax -> 512x512xnum_classes

The skip connections concatenate encoder features with decoder features at matching resolutions, preserving spatial detail for precise boundary segmentation of roads, vehicles, pedestrians, and other scene elements.

Part 2: Recurrent Neural Networks (RNNs)

Q11. Why are traditional feedforward neural networks not suitable for sequence data? How do RNNs solve this? Give two real-world examples.

Traditional feedforward neural networks (FFNNs) process each input independently with no memory of previous inputs. They expect fixed-size input vectors and produce fixed-size outputs. This makes them unsuitable for sequence data where order, context, and temporal dependencies matter.

Why FFNNs fail for sequences:

- Fixed input size: FFNNs cannot handle variable-length sequences like sentences of different lengths.
- No temporal memory: Each input is processed in isolation. The network cannot remember that 'The cat sat...' occurred when processing 'on the mat'.
- Parameter explosion: To process sequences by flattening, FFNNs would need enormous input dimensions with separate weights for each position.

How RNNs solve this: RNNs introduce a hidden state (h_t) that acts as memory. At each time step, the RNN updates its hidden state using both the current input (x_t) and the previous hidden state (h_{t-1}). This creates a recurrent connection that allows information to persist across time steps. The same weights are shared across all time steps, enabling processing of variable-length sequences.

Formula: $h_t = \tanh(W_h * h_{t-1} + W_x * x_t)$

Real-world examples:

- Natural Language Processing: Sentiment analysis, machine translation, and text generation require understanding word order and context.
- Time Series Prediction: Stock price forecasting, weather prediction, and sensor data analysis require modeling temporal dependencies.

Q12. Draw a basic RNN unit and explain Input x_t , Hidden state h_t , Output o_t . What is the role of the hidden state?

A basic RNN unit has the following structure: At each time step t , the unit receives an input x_t and the previous hidden state h_{t-1} . It computes the new hidden state h_t and produces an output o_t .

Input x_t : The input at time step t . In NLP, this is typically the embedding vector of the t -th word in a sentence. In time series, it is the value at time t .

Hidden state h_t : The memory of the network at time t . It encodes information about all inputs seen so far ($x_1, x_2, \dots, x_t$). Computed as: $h_t = \text{activation}(W_h * h_{t-1} + W_x * x_t + b_h)$

Output o_t : The prediction at time t . Computed from the hidden state: $o_t = W_o * h_t + b_o$. For classification, a softmax may be applied to o_t .

Role of the hidden state: The hidden state serves as the network's memory. It compresses all past information into a fixed-size vector that is updated at each step. When the RNN reads a new input, it combines this new information with its existing memory to form an updated understanding. This allows the RNN to maintain context - for example, remembering that a singular noun appeared earlier to determine correct verb conjugation later.

Q13. Given $h_t = \tanh(W_h * h_{t-1} + W_x * x_t)$, with $h_{t-1}=0.5$, $x_t=1$, $W_h=0.8$, $W_x=0.2$. Compute (a) input to activation, (b) h_t , (c) What this value represents.

Given: $h_{t-1} = 0.5$; $x_t = 1$; $W_h = 0.8$; $W_x = 0.2$

(a) Input to Activation (pre-activation z):

$$\begin{aligned} z &= W_h * h_{t-1} + W_x * x_t \\ z &= (0.8 \times 0.5) + (0.2 \times 1) \\ z &= 0.4 + 0.2 = 0.6 \end{aligned}$$

(b) Compute h_t :

$$\begin{aligned} h_t &= \tanh(z) = \tanh(0.6) \\ \text{Using } \tanh(0.6) &= (e^{0.6} - e^{-0.6}) / (e^{0.6} + e^{-0.6}) \\ e^{0.6} &= 1.8221, \quad e^{-0.6} = 0.5488 \\ \tanh(0.6) &= (1.8221 - 0.5488) / (1.8221 + 0.5488) = 1.2733 / 2.3709 = \\ &0.5370 \\ h_t &= 0.537 \text{ (approximately)} \end{aligned}$$

(c) What this value represents: The value $h_t = 0.537$ represents the updated hidden state of the RNN after processing the current input. It encodes the combined information from the previous memory ($h_{t-1} = 0.5$) and the current input ($x_t = 1$). The tanh activation squashes the value between -1 and 1, preventing explosion and introducing non-linearity. This hidden state will be passed to the next time step as h_t , allowing information to flow through the sequence.

Q14. Explain what it means to unroll an RNN through time. Why is unrolling important for training? How does it relate to backpropagation?

Unrolling an RNN through time means expanding the recurrent structure across all time steps to create a feedforward-like computational graph. Instead of showing a single RNN cell with a loop, unrolling reveals copies of the same cell at each time step, connected by the hidden state.

Why unrolling is important for training:

- **Computational graph:** Unrolling creates an explicit directed acyclic graph (DAG) that shows exactly how gradients flow from the loss back to each parameter at each time step.
- **Shared parameters:** When unrolled, it becomes clear that the same weight matrices (W_h , W_x , W_o) are used at every time step. Gradients from all time steps must be summed for these shared parameters.
- **Variable length handling:** Unrolling allows processing sequences of different lengths - we simply unroll for as many steps as the sequence length.

Relation to backpropagation: The unrolled RNN enables a variant of backpropagation called Backpropagation Through Time (BPTT). In BPTT, the loss gradient at the final output is propagated backward through the unrolled graph. Because the hidden states h_t are connected to h_{t-1} , gradients flow not only through the direct connections but also through the recurrent hidden state connections. This means the gradient at time t depends on all previous time steps, creating long-term dependency learning.

Q15. Explain the following RNN input-output structures: one-to-one, one-to-many, many-to-one, many-to-many. Give one example for each.

One-to-One: Single input produces single output. This is equivalent to a standard feedforward network. Example: Image classification - one image maps to one class label.

One-to-Many: Single input produces a sequence of outputs. The RNN receives one input and generates multiple outputs autoregressively. Example: Image captioning - one image is processed by a CNN encoder, then an RNN decoder generates a sentence describing the image.

Many-to-One: A sequence of inputs produces a single output. The entire sequence is processed, and only the final hidden state is used for prediction. Example: Sentiment analysis - a sequence of words is read, and the final output classifies the sentiment as positive or negative.

Many-to-Many (synchronized): A sequence of inputs produces a sequence of outputs, with input and output lengths equal. Example: Named Entity Recognition (NER) - each word in the input sentence gets a label (PERSON, LOCATION, ORGANIZATION, etc.).

Many-to-Many (encoder-decoder): Input sequence of length N produces output sequence of length M , where N and M may differ. Example: Machine translation - an English sentence (N words) is translated to a French sentence (M words).

Q16. What is Backpropagation Through Time (BPTT)? How is it different from standard backpropagation? Why is it computationally expensive?

Backpropagation Through Time (BPTT) is the algorithm used to train RNNs by computing gradients of the loss with respect to the network's parameters. It extends standard backpropagation to handle the temporal structure of RNNs.

How BPTT works: The RNN is first unrolled through all time steps, creating a computational graph. The forward pass computes hidden states and outputs at each step. The backward pass starts from the final loss and propagates gradients backward through the unrolled graph, through both the output connections and the recurrent hidden state connections.

Difference from standard backpropagation:

- Temporal unfolding: Standard backpropagation operates on a fixed DAG. BPTT must handle a graph that is unrolled across time steps.
- Shared parameter gradients: In standard BP, each parameter has its own gradient. In BPTT, the same weight matrices appear at every time step, so their gradients must be summed across all time steps.
- Long-term gradient flow: BPTT must propagate gradients through many time steps via the hidden state connections, which can lead to vanishing or exploding gradients.

Why BPTT is computationally expensive:

- Memory: The unrolled graph requires storing all hidden states and intermediate activations for the entire sequence during the forward pass.
- Time complexity: For a sequence of length T , BPTT requires $O(T)$ operations per parameter. Long sequences multiply the computational cost.
- Truncation: In practice, BPTT is often truncated to a fixed window (e.g., 20-50 steps) to manage computation, but this limits the model's ability to learn very long-term dependencies.

Q17. Explain the vanishing gradient problem in RNNs. Why does it occur? What is its effect on learning long-term dependencies?

The vanishing gradient problem occurs when gradients become exponentially small as they are propagated backward through many time steps during BPTT. This makes it difficult for the RNN to learn long-term dependencies.

Why it occurs: During BPTT, the gradient of the loss with respect to the hidden state at time t involves multiplying by the weight matrix W_h and the derivative of the activation function repeatedly. For $\tanh$, the derivative is at most 1. For a sequence of length T , the gradient involves terms like $(W_h^T)^T$. If the largest eigenvalue of W_h is less than 1, these terms shrink exponentially with T . Even with eigenvalues near 1, repeated multiplication by values less than 1 causes decay.

Mathematically: dL/dh_t contains factors of $\text{product}(W_h * \text{diag}(\tanh'(z_i)))$ for $i=t+1$ to T

Effect on learning long-term dependencies:

- Short-term memory only: Gradients from distant time steps vanish, so the RNN only updates based on recent inputs.
- Failure to connect distant events: In a sentence like 'The cat, which was very hungry and had been wandering for hours, sat on the mat', the RNN may fail to link 'cat' (subject) with 'sat' (verb) due to the long distance.
- Poor performance on long sequences: Tasks requiring understanding of long-range context (document-level sentiment, long document classification) suffer significantly.

Q18. What is the exploding gradient problem? How does it affect training?

Suggest one method to handle it.

The exploding gradient problem is the opposite of vanishing gradients - gradients become exponentially large when propagated backward through many time steps. This causes unstable training with massive weight updates.

How it affects training:

- Numerical overflow: Gradients become NaN (not a number) due to floating-point overflow.
- Unstable updates: Weight updates are so large that parameters jump to extreme values, destroying learned representations.
- Loss divergence: The training loss may suddenly spike to infinity instead of converging.
- Oscillation: Large gradients cause the optimizer to overshoot optimal parameter values repeatedly.

Mathematical cause: Similar to vanishing gradients, but occurs when the largest eigenvalue of W_h is greater than 1. The product $(W_h^T)^T$ grows exponentially with sequence length T .

Solution - Gradient Clipping: The most common solution is gradient clipping. When the gradient norm exceeds a threshold, it is rescaled to have that threshold as its norm. This prevents any single gradient from being too large while preserving the gradient direction.

If $\| \text{gradient} \| > \text{threshold}$: $\text{gradient} = \text{threshold} * \text{gradient} / \| \text{gradient} \|$

Q19. Why were LSTMs introduced? What problem do they solve? Briefly explain the idea of memory cell and gates.

Long Short-Term Memory (LSTM) networks were introduced by Hochreiter and Schmidhuber in 1997 specifically to address the vanishing gradient problem in standard RNNs. They enable learning of long-term dependencies by design.

Problem solved: LSTMs solve the vanishing gradient problem through a carefully designed gating mechanism that allows information to flow through the network unchanged over long sequences. The key innovation is the cell state - a conveyor belt that runs through the entire chain with only linear interactions.

Memory Cell (Cell State C_t): The cell state is the internal memory of the LSTM. It runs straight through the chain with minimal interactions - only element-wise additions and multiplications. Information can be added to or removed from the cell state via gates. Because the cell state path is mostly linear (just multiplications by values close to 1), gradients can flow backward through many time steps without vanishing.

Gates: Gates are sigmoid-activated neural networks that control information flow. They output values between 0 (block) and 1 (allow through).

- Forget Gate (f_t): Decides what information to discard from the cell state. $f_t = \text{sigmoid}(W_f * [h_{t-1}, x_t] + b_f)$
- Input Gate (i_t): Decides what new information to store in the cell state. $i_t = \text{sigmoid}(W_i * [h_{t-1}, x_t] + b_i)$
- Output Gate (o_t): Decides what to output based on the cell state. $o_t = \text{sigmoid}(W_o * [h_{t-1}, x_t] + b_o)$

Cell state update: $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$

Hidden state: $h_t = o_t * \tanh(C_t)$

Q20. Design an RNN-based model for sentiment analysis. Include input representation, RNN processing, final output layer, and RNN structure type.

Architecture for Sentiment Analysis using RNN:

Input Representation:

- Text tokenization: Split the input sentence into words/tokens.
- Word embeddings: Map each token to a dense vector using a pre-trained embedding matrix (e.g., Word2Vec, GloVe) or learned embeddings. For a sentence of length T with embedding dimension d , this produces a sequence of vectors: $[x_1, x_2, \dots, x_T]$ where each x_i is in $\mathbb{R}^d$.

RNN Processing (Many-to-One structure):

- Use an LSTM or GRU layer to process the sequence. At each time step t : $h_t = \text{LSTM}(x_t, h_{t-1})$
- The LSTM updates its cell state and hidden state at each word, accumulating semantic understanding.
- After processing all T words, the final hidden state h_T contains a compressed representation of the entire sentence.

Final Output Layer:

- Pass h_T through a fully connected layer: $z = W * h_T + b$
- Apply softmax for classification: $y_{\text{hat}} = \text{softmax}(z)$
- For binary sentiment: output dimension = 2 (positive, negative)
- For fine-grained: output dimension = 5 (very negative to very positive)

RNN Structure Type: Many-to-One - the model reads a sequence of words (many inputs) and produces a single sentiment label (one output). Using bidirectional LSTM (processing the sentence both forward and backward) often improves performance by capturing context from both directions.

Assignment 2: Convolutional Neural Networks

Q1. An input image of size 32 x 32 passes through a convolution layer with filter=5x5, stride=1, padding=0. Calculate (a) output feature map size, (b) final output dimension with 10 filters.

Given: Input = 32 x 32; Filter = 5 x 5; Stride = 1; Padding = 0; Number of filters = 10

(a) Output Feature Map Size:

Formula: $\text{Output} = (W - F + 2P) / S + 1$
 $\text{Output} = (32 - 5 + 2(0)) / 1 + 1 = 27 / 1 + 1 = 28$
 Output feature map size = 28 x 28

(b) Final Output Dimension with 10 filters:

Each filter produces one feature map, so 10 filters produce 10 channels.
 Final output dimension = 28 x 28 x 10

Q2. Given input 28 x 28, filter 3x3, stride=1, padding=1. (a) Compute output size, (b) Explain why padding is used in CNNs.

Given: Input = 28 x 28; Filter = 3 x 3; Stride = 1; Padding = 1

(a) Output Size:

$\text{Output} = (W - F + 2P) / S + 1$
 $\text{Output} = (28 - 3 + 2(1)) / 1 + 1 = (28 - 3 + 2) + 1 = 28$
 Output size = 28 x 28

(b) Why padding is used in CNNs:

- Preserve spatial dimensions: With 'same' padding ($P = (F-1)/2$), the output has the same size as the input, making it easier to design deep networks.
- Prevent information loss at borders: Without padding, corner pixels are sampled fewer times than center pixels, causing border information to be underrepresented.
- Enable deeper networks: Without padding, each conv layer shrinks the feature map, limiting network depth.
- Maintain feature map size: As shown above, padding=1 with 3x3 filter preserves the 28x28 dimension.

Q3. Explain parameter sharing in CNNs. Why is it important compared to fully connected layers? Discuss impact on parameters, overfitting, and computation cost.

Parameter sharing in CNNs means that the same filter (kernel) is used across all spatial positions of the input. A single filter slides over the entire image, applying the same weights at every location. This is also called weight tying.

Importance compared to fully connected layers: In a fully connected layer, every input unit connects to every output unit with a unique weight. For a 100x100 image with 1000 outputs, this requires 10,000,000 weights. A CNN with a 5x5 filter producing 1000 feature maps needs only $5 \times 5 \times 1000 = 25,000$ weights - a 400x reduction.

Impact on number of parameters: Parameter sharing dramatically reduces the number of trainable parameters. A conv layer with $F \times F$ filter, C_{in} input channels, and C_{out} output channels has $(F \times F \times C_{in} + 1) \times C_{out}$ parameters regardless of input spatial size. Without sharing, parameters would grow with input dimensions.

Impact on overfitting: Fewer parameters mean less risk of overfitting. The model is forced to learn features that are translation-invariant (detectable anywhere in the image) rather than memorizing position-specific patterns. This acts as a strong inductive bias for image data.

Impact on computation cost: Parameter sharing enables efficient computation using convolution operations, which are highly optimized on GPUs. Instead of computing separate dot products for each position, convolutions use fast Fourier transforms or matrix multiplications (im2col) for parallel processing.

Q4. A convolution layer has input channels=3, filters=16, filter size=3x3. Calculate (a) total number of weights, (b) total parameters including bias.

Given: Input channels (C_{in}) = 3; Number of filters = 16; Filter size = 3×3

(a) Total Number of Weights:

```
Each filter has size:  $F \times F \times C_{in} = 3 \times 3 \times 3 = 27$  weights
Total weights = Number of filters x Weights per filter
Total weights =  $16 \times 27 = 432$ 
```

(b) Total Parameters Including Bias:

```
Each filter has 1 bias term
Total bias parameters = 16
Total parameters = Total weights + Total bias =  $432 + 16 = 448$ 
```

Q5. Given input feature map: $[[1,3,2,1],[4,6,5,2],[7,8,9,3],[1,2,0,4]]$. Apply 2×2 max pooling with stride=2. Compute output feature map.

Given Input (4x4):

```
Row 0: 1  3  2  1
Row 1: 4  6  5  2
Row 2: 7  8  9  3
Row 3: 1  2  0  4
```

Pooling: 2×2 kernel, stride=2

```
Top-left quadrant (rows 0-1, cols 0-1): max(1,3,4,6) = 6
Top-right quadrant (rows 0-1, cols 2-3): max(2,1,5,2) = 5
Bottom-left quadrant (rows 2-3, cols 0-1): max(7,8,1,2) = 8
Bottom-right quadrant (rows 2-3, cols 2-3): max(9,3,0,4) = 9
```

Output Feature Map (2×2):

```
| 6  5 |
| 8  9 |
```

Q6. Consider a CNN: Input $32 \times 32 \times 3$, Conv: 8 filters (3×3), stride 1, padding 1, MaxPool: 2×2 stride 2, FC: 10 classes. Compute (a) conv output, (b) pool output, (c) explain flattening, (d) final activation.

(a) Output after Convolution:

```
Output = (32 - 3 + 2(1)) / 1 + 1 = (32 - 3 + 2) + 1 = 32
Conv output = 32 x 32 x 8
```

(b) Output after Pooling:

```
Output = 32 / 2 = 16
Pool output = 16 x 16 x 8
```

(c) Flattening explanation: Before the fully connected layer, the 3D feature map ($16 \times 16 \times 8 = 2048$ values) is flattened into a 1D vector of 2048 elements. This is necessary because FC layers expect 1D input. Each of the 2048 values becomes one input to the FC layer. The flattening preserves all values - it only reshapes the tensor.

(d) Final activation: Softmax is used at the final layer because this is a multi-class classification problem. Softmax converts the 10 output logits into a probability distribution where all values are positive and sum to 1. This allows interpreting outputs as class probabilities. The predicted class is the one with the highest probability.

```
Softmax( $z_i$ ) =  $e^{(z_i)} / \sum(e^{(z_j)})$  for  $j = 1$  to 10
```

Q7. An input image 64x64 passes through convolution with filter=7x7, stride=2, padding=0. (a) Calculate output size, (b) How stride affects spatial resolution.

Given: Input = 64 x 64; Filter = 7 x 7; Stride = 2; Padding = 0

(a) Output Size:

$$\begin{aligned}\text{Output} &= (W - F + 2P) / S + 1 \\ \text{Output} &= (64 - 7 + 0) / 2 + 1 = 57 / 2 + 1 = 28.5 + 1 = 29.5 \\ \text{Since we take the floor: Output} &= 29 \times 29\end{aligned}$$

(b) How stride affects spatial resolution: Stride determines how far the filter moves between consecutive applications. With stride=1, the filter moves one pixel at a time, producing output nearly the same size as input (minus filter size). With stride=2, the filter skips every other pixel, effectively downsampling the feature map by a factor of 2. Larger strides produce smaller outputs more quickly but may miss fine details because not all input positions are sampled. Stride acts as a learnable downsampling alternative to pooling.

Q8. Why are non-linear activation functions (like ReLU) used after convolution layers? What happens if only linear operations are used?

Non-linear activation functions like ReLU (Rectified Linear Unit: $f(x) = \max(0, x)$) are essential after convolution layers because they introduce non-linearity into the network. Without non-linearities, a deep neural network would be equivalent to a single linear transformation, regardless of depth.

Why ReLU is used:

- **Non-linearity:** Real-world data is rarely linearly separable. ReLU allows the network to learn complex, non-linear decision boundaries.
- **Computational efficiency:** ReLU is extremely cheap to compute - just a thresholding operation.
- **Mitigates vanishing gradients:** Unlike sigmoid/tanh, ReLU does not saturate for positive inputs, allowing gradients to flow more freely during backpropagation.
- **Sparsity:** ReLU outputs zero for negative inputs, creating sparse activations which can improve computational efficiency.

What happens with only linear operations: If all layers were linear (convolution without activation), a network with any number of layers would be mathematically equivalent to a single linear transformation. For example, if Layer 1 computes $y = W1*x + b1$ and Layer 2 computes $z = W2*y + b2$, then $z = W2*(W1*x + b1) + b2 = (W2*W1)*x + (W2*b1 + b2) = W'*x + b'$. This is just a single linear layer. The network would be unable to learn complex patterns, making it no more powerful than logistic regression.

CNNs need non-linearities to learn hierarchical features: edges in early layers, textures in middle layers, and object parts in deep layers. Without non-linear activations, this hierarchical representation would be impossible.

Q9. An RGB image (3 channels) passes through convolution with 5 filters, 3x3, stride=1, padding=0. (a) Depth of each filter, (b) Total parameters including bias, (c) Output dimension for 32x32x3 input.

Given: Input channels = 3; Filters = 5; Filter size = 3 x 3; Stride = 1; Padding = 0; Input size = 32 x 32 x 3

(a) Depth of Each Filter:

Filter depth = Number of input channels = 3
Each filter has shape: 3 x 3 x 3

(b) Total Parameters Including Bias:

Weights per filter = 3 x 3 x 3 = 27
Total weights = 5 x 27 = 135
Bias per filter = 1
Total bias = 5 x 1 = 5
Total parameters = 135 + 5 = 140

(c) Output Dimension:

Output spatial = $(32 - 3 + 0) / 1 + 1 = 30$
Output channels = Number of filters = 5
Output dimension = 30 x 30 x 5

Q10. (a) Describe the complete CNN pipeline for image classification. Explain each stage. (b) What does a feature map represent? How do early vs deep layers differ?

(a) Complete CNN Pipeline for Image Classification:

- **Input Image:** The raw image tensor (e.g., 224 x 224 x 3 for RGB). Pixel values are typically normalized to [0, 1] or [-1, 1].
- **Convolution Layers:** Apply learnable filters to detect local patterns. Each filter produces a feature map highlighting where its pattern appears. Multiple filters detect different features.
- **Activation Functions (ReLU):** Apply non-linearity after each convolution. Set negative values to zero, introducing sparsity and enabling non-linear learning.

- **Pooling Layers:** Downsample feature maps using max or average pooling. Reduces spatial dimensions, provides translation invariance, and decreases computation.
- **Fully Connected Layer:** After flattening the final feature maps, FC layers combine all features to make the final classification decision. They learn global patterns across all locations and channels.
- **Output (Softmax):** Converts logits to class probabilities. The class with the highest probability is the predicted label.

(b) Feature Maps and Layer Differences:

A feature map represents the spatial response of a filter across the input. Each value in a feature map indicates how strongly the filter's pattern matches at that location.

Early layers: Detect low-level, generic features like edges, corners, color gradients, and textures. These features are small and local. The filters are relatively simple and transferable across different tasks.

Deep layers: Detect high-level, complex features like object parts (eyes, wheels), complete objects (faces, cars), or scenes. These features are larger, more abstract, and task-specific. Deep layers combine low-level features from earlier layers to form semantic understanding.

For example, in a face detection CNN, early layers might detect edges and simple textures, middle layers might detect eyes and noses, and deep layers might detect complete face pattern

